

Programação OO

Separando Responsabilidades

Responsabilidades

- Mantenha suas classes focadas mantendo as responsabilidades bem definidas;
- Imagine uma empresa em que o Presidente é responsável por tudo. Isto não é uma boa idéia, conforme o negócio cresce ele precisará delegar responsabilidades para outras pessoas;
- O mesmo ocorre com nossa aplicação, conforme ela cresce, fica difícil de mantê-la se as responsabilidades não estiverem bem definidas;



Separando Responsabilidades

- Avalie se a classe deve realmente fazer tudo aquilo que foi especificado, ou se ela deve ser fragmentada em classes menores;
- Este princípio tem por objetivo fragmentar a aplicação em partes em que cada parte fique responsável por uma "preocupação" em específico;
- Minimiza o Acoplamento;
- Maximiza a Coesão;
- Simplifica a Manutenção;
- Facilita o Teste;



Minimizar o Acoplamento

Acoplamento é o grau de dependência de uma classe para com outras ou com recursos externos;

Quando menos dependências uma classe tiver, mais fácil é de escrever rotinas de teste, manter e atualizar com o tempo;

Se houverem muitas dependências em uma classe, considere mover algumas de suas dependências para outra classe;

Customer

- Name
- Email address
- Home address
- Work address
- Validate()
- Retrieve()
- Save()

Product

- Product name
- Description
- Current price
- Validate()
- Retrieve()
- Save()

Order

- Customer
- Order date
- Shipping address
- Order items
- Validate()
- Retrieve()
- Save()

Order Item

- Product
- Quantity
- Purchase price
- Validate()
- Retrieve()
- Save()

Minimizar o Acoplamento

- Por exemplo, as classes definidas possuem métodos responsabilidades que são da camada de acesso a dados.
- Minimiza o risco de uma mudança nesta classe afetar outras.

Maximizar Coesão

- É uma medida de quão relacionado tudo na classe é ao propósito da classe;
- Se houverem métodos ou atributos que não são relacionados ao propósito da classe eles devem ser movidos para outra classe;
- Por exemplo, o endereço de entrega. Ele possui muitos atributos e informações que não são especificamente do pedido, nem do consumidor, mas do endereço em sua essência;
- Assim, poderíamos mover a especificação de endereço para uma classe definida para ela;



BORDÕES

- YAGNI – You Aren't Gonna Need It
 - Não adicione código em suas classes até que eles sejam realmente necessários;
- KISS – Keep It Simple Stupid
 - Mantenha as coisas simples. Se está complicado provavelmente está errado;
- DRY – Don't Repeat Yourself
 - Escreva classes e métodos de modo que eles possam ser reaproveitados;

KISS

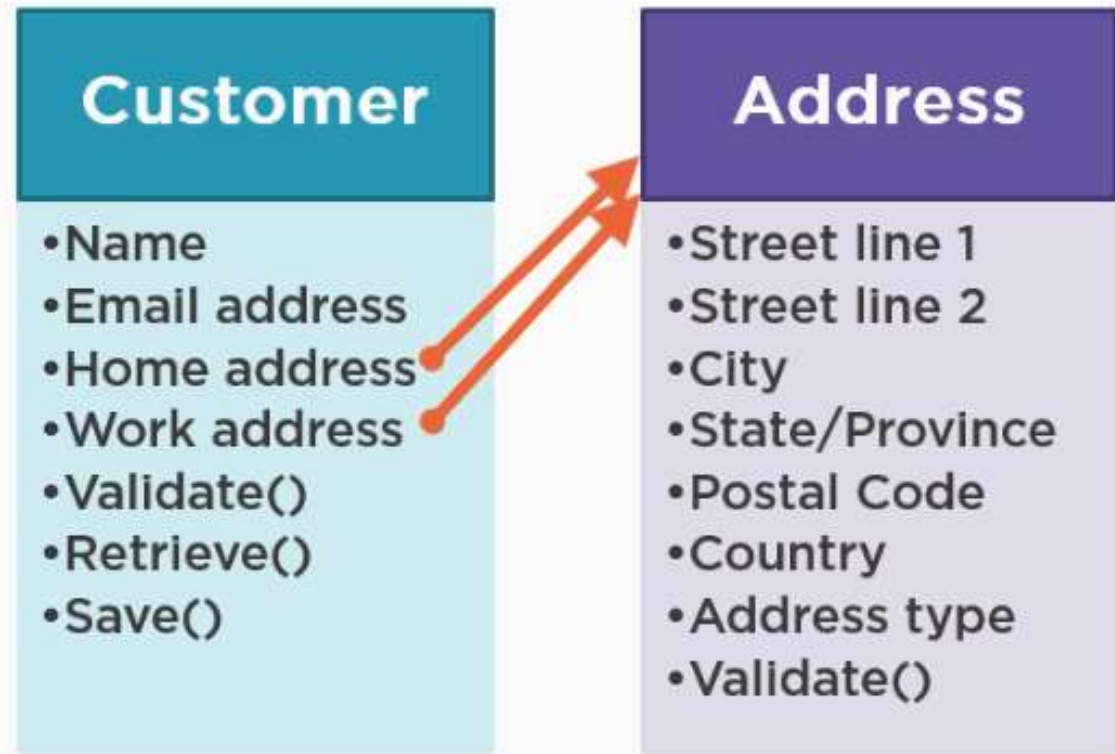
Keep It Simple, Stupid!

YAGNI

You Aren't Gonna Need It

DRY

Don't Repeat Yourself



Podemos desmembrar a classe Customer com uma especificação mais detalhada para Address

Separando Responsabilidades





E Quanto aos Métodos?

- Os métodos `Retrieve()` e `Save()` tem mais relação com a Camada de Dados.
- Podemos separá-los em uma classe específica para lidar com o Repositório de Dados;

Podemos aplicar o conceito do Padrão de Design Repositório para as demais classes;

O Padrão Repositório



Classe Address

```
public class Address
{
    public Address()
    {
    }

    public Address(int addressId)
    {
        AddressId = addressId;
    }

    public int AddressId { get; private set; }
    public int AddressType { get; set; }
    public string City { get; set; }
    public string Country { get; set; }
    public string PostalCode { get; set; }
    public string State { get; set; }
    public string StreetLine1 { get; set; }
    public string StreetLine2 { get; set; }
}
```

```
/// <summary>
/// Validates the address data.
/// </summary>
/// <returns></returns>
public bool Validate()
{
    var isValid = true;

    if (PostalCode == null) isValid = false;

    return isValid;
}
```

```

public Customer Retrieve(int customerId)
{
    // Create the instance of the Customer class
    // Pass in the requested id
    Customer customer = new Customer(customerId);

    // Code that retrieves the defined customer

    // Temporary hard-coded values to return
    // a populated customer
    if (customerId == 1)
    {
        customer.EmailAddress = "fbaggins@hobbiton.me";
        customer.FirstName = "Frodo";
        customer.LastName = "Baggins";
    }
    return customer;
}

/// <summary>
/// Saves the current customer.
/// </summary>
/// <returns></returns>
public bool Save(Customer customer)
{
    // Code that saves the passed in customer

    return true;
}

```

Repositórios

- Um repositório cuida da interação com a fonte de dados;
- No projeto de Biblioteca de Classes UNOESC.BL, adicione uma nova classe C# com o nome CustomerRepository.cs e inclua os métodos no corpo da classe:

Classe Customer

- Agora você pode remover os métodos que estão especificados na classe Customer.cs, pois movemos a implementação para CustomerRepository.cs

Demais Classes

- Agora você deve refatorar as demais classes com o mesmo padrão de projeto que definimos para Repository;
 - ProductRepository.cs
 - OrderRepository.cs
- Não esqueça de remover os métodos das classes originais:
 - Product
 - Order
- Ainda não altere a classe OrderItem, depois trataremos dos relacionamentos entre classes!